

Document #: P3237R1
Date: 2024-10-16
Project: Programming Language C++
Audience: SG21
Reply-to:
Andrei Zissu <andrziss@gmail.com>

Matrix Representation of Contract Semantics

Contents

Contents
Introduction
Summary of Proposed Changes
Motivation
Proposal
Impact of the Changes
Q&A
Wording
References

Revision History

Revision 0 (April 2024 Mailing)

- Initial revision.

Revision 1 (October 2024 Mailing)

- Properties as bitmasks rather than bit fields.
- Updated some sections.
- Added a code example (link to godbolt).
- Semantic properties are now always represented as a single boolean value.
- Added a questions for SG21 section.
- Added a Q&A paragraph on unsuccessful prior art.

Introduction

Contract semantics, as proposed in [P2900R5], already comprise a set of 4 semantics - *ignore*, *enforce*, *observe*, and *quick_enforce* (the former “Louis semantic”). Several new semantics are quite likely to be proposed in the foreseeable future. As can be currently seen with the “Louis semantic” which took a while plus some considerable discussion to finally be named *quick_enforce*, deciding upon proper naming of such semantics often incurs considerable difficulties. This is spurred by the justified fear of an increasing number of contract semantics posing ever more challenges to the intuitive grasp of their meaning by the C++ community and even by WG21 members.

We therefore propose a different way of defining contract semantics, which would address the issues described above and even the design of new contract semantics.

Summary of Proposed Changes

We are proposing that contract semantics (as proposed in [P2900R5]) be defined in terms of separate properties, and that they be represented as a matrix comprising said properties. While officially orthogonal - as seen below, some dependencies are expected to exist between them, thus reducing the number of eligible combinations.

Optionally, we also propose redefining the permissible values of the `contract_semantic` field of class `contract_violation` so as to incorporate the new properties-based representation via combinations of power-of-2 values rather than the current sequential enum..

Motivation

The contracts status quo represented by [P2900R5] currently includes 4 semantics: *ignore*, *enforce*, *observe* and *quick_enforce*. An *assume* semantic is already viewed as a likely post-MVP proposal. Additional semantics may be proposed, such as the tentative *terminate* semantic in [P3205R0].

Describing such a multitude of semantics and comparing them is quickly becoming a challenge. [P3205R0] tackles that in a way that seems natural and called for – a matrix (this example is from an early version of that paper, which for demonstration purposes is immaterial):

semantic	checks	calls handler	assumed after	terminates	proposed
assume	no	no	yes	no	no
ignore	no	no	no	no	[P2900R5]
“Louis”	yes	no	yes	trap-ish	TODO
terminate	yes	no	yes	<code>std::terminate</code>	here
observe	yes	yes	no	no	[P2900R5]
ensure	yes	yes	yes	<code>std::abort-ish</code>	[P2900R5]

(Note: *enforce* in the above table is erroneously referred to as *ensure*.)

Describing contract semantics in this manner affords us a bird's eye view of all their salient properties, as well as allowing us to spot missing properties which should be properly specified for each new proposed semantic.

In addition to allowing easier reasoning about the differences between various semantics, such a description would also allow us to consider the need for new semantics. This could be done by first determining dependencies between some matrix columns, and from there new semantics allowed by those dependencies might fall out.

For example, let's first determine some dependencies:

!checks implies *!calls_handler* – Not checking the predicate implies the violation handler will never be called. Similarly: *!checks* implies *!terminates*

terminates implies *assumed_after* – A terminating semantic (with whatever termination means) allows the compiler to optimize function code following the contract assertion under the assumption that it will only be executed in-contract. This assumption holds even in semantics (such as *enforce*) which allow termination to be bypassed, e.g. by throwing an exception. (Note: One could imagine future terminating semantics which may challenge this assumption, e.g. conditional or delayed termination.)

Such dependencies help us reduce the combinatorial explosion of legal matrix combinations and therefore of possible and sensible semantics. Whatever remains after eliminating illegal combinations may inspire proposals for new semantics.

This paper does not propose any particular set of matrix columns, i.e. contract semantic properties. If this proposal is adopted, we will have a principle by which such separate properties may be proposed (and possibly extended later). Thus, this paper can be viewed as mainly a policy proposal.

Proposal

We propose that henceforth contract semantics be described in terms of separate properties (unspecified as to their actual content in this proposal). The full list of available semantics will be visualized as a matrix, with each column representing a semantic property and each row a contract semantic. Each property will be described in terms of a single boolean flag with `true` and `false` values.

Going back to the earlier matrix example taken from [P3205R0]: *checks*, *calls_handler* and *assumed_after* would each be represented as a single property. *Terminate* lists 4 possible states, which would be represented by the combination of 2 boolean flags, therefore it would need to be defined as 2 properties. *Proposed* is a comments column, not represented as a property.

We further propose that named semantics would still be listed in the standard, but their description in code will be via a set of `constexpr` (power of 2) values. (Presumably this could also facilitate command line usage,

as compiler flags would not necessarily have to be provided individually for each separate contract semantic property.) Thus, the current [P2900R8] status quo could be represented as in this code example:

<https://compiler-explorer.com/z/684TMYPo8>

In terms of library facilities, we *optionally* propose mandating that the `contract_semantic` enum field of class `contract_violation` will contain only power-of-2 values, making it bitmask-friendly. This would enable easy inspection/specification of multiple properties en masse. This would require standardizing the underlying type of the `contract_semantic` enum - we believe a 32 or 64 bit integer type would more than suffice for any future expansion, given that new contract semantic properties are expected to be few and far between (unlike new contract semantics, which are combinations thereof). With a 64-bit underlying type, we could reserve the upper 32 bits for vendor extensions - this may only prove unsatisfactory if different vendors may require interactions, thus needing unique universal property values.

With these 2 optional additions in place, the contract semantics listed in the [P2900R5] status quo) could be described via `constexpr` definitions. We would currently informally propose the following, semantic aliases (as listed in the code example mentioned above):

```
CONTRACT_SEMANTIC(ignore);
CONTRACT_SEMANTIC(observe, evaluates_predicate);
CONTRACT_SEMANTIC(enforce, evaluates_predicate, calls_violation_handler,
enforces, terminates);
CONTRACT_SEMANTIC(quick_enforce, evaluates_predicate, enforces,
terminates);
```

Impact of the Changes

- No impact on current code, as contracts are not yet part of C++.
- Possible changes in the permitted values of the `contract_semantic` enum as proposed in [P2900R5].

Q&A

Wouldn't this create a huge combinatorial matrix and only complicate things?

Not if we reign it in, by means of carefully defining inter-column dependencies as described in this paper. Thus we would never populate the matrix with semantics made up of property combinations not permissible as per the defined dependencies.

Didn't similar past proposals fail?

Well, yes and no. Several proposals mentioned in [\[P3227R0\]](#) indeed failed to gain traction. However, they proposed contract semantic properties as first class entities, whereas we propose them as building blocks of semantic aliases. Whereas we have decided for now to present semantic aliases as an optional addition to this paper, the fact is that having them as first class entities and precluding any other property combinations as non-standard (though free to experiment with outside the standard) would most likely solve the main stumbling block of previous proposals by limiting the number of standard semantic to the same number presented by the current [\[P2900R5\]](#) status quo.

Wouldn't the proposed aliases bring us back to the naming conundrums described as motivation for this proposal?

Well, yes. However, having control over the number of legitimate property combinations (a.k.a semantics) is a stronger motivation. Having said that, having each alias/semantic directly defined as its set of properties will greatly aid in understanding their meaning, writing code referring to individual properties rather than full semantics, and possibly experimentation with new semantics.

Questions for SG21

- Do we agree with the general direction presented in this paper?
- Are we interested in redefining `contract_violation::contract_semantic` as a set of power-of-2 boolean combinations?
- Which underlying type would we like to choose, and how do we divide it between standard and vendor-specific properties? Do the latter need to be universally unique?

Wording

To be added later, if needed.

References

[P2900R5] Joshua Berne, Timur Doumler, Andrzej Krzemiński. 2024-02-15. *Contracts for C++*.
<https://wg21.link/p2900r5>

[P3205R0] Gašper Ažman, Jeff Snyder, Andrei Zissu. 2024-04-15. *Throwing from a noexcept function should be a contract violation*.
<https://isocpp.org/files/papers/P3205R0.pdf>

[P3227R0] Gašper Ažman, Timur Doumler. 2024-10-16. *Fixing the library API for contract violation handling*.
<https://isocpp.org/files/papers/P3227R0.pdf>